

Design and Implementation of the JAVA-- language Compiler
Checkpoint 3

Compilers - L.EIC026 - 2025/2026

João Bispo

Past contributions of
Tiago Carvalho, Alexandre Abreu, José Ferreira,
Nuno Cardoso, Pedro Gonçalo Correia, Tiago Santos

University of Porto/FEUP
Department of Informatics Engineering

version 1.0, May 2026

Contents

1	Generate Jasmin code	2
1.1	Interfaces	2
1.2	Always generate a public class	2
1.3	Instruction Selections	3
1.4	.limit directives	3
1.5	When in doubt, reverse engineer it	3
2	Checklist for CP3	4
2.1	Checklist for Core Features	4
2.1.1	Declarations	4
2.1.2	Statements	4
2.1.3	Entities and Operations	4
2.1.4	Calls	5
2.1.5	Arrays	5
2.2	Checklist for Extensions	5
2.2.1	Declarations	5
2.2.2	Statements	5
2.2.3	Entities and Operations	6
2.2.4	Calls	6
2.2.5	Arrays	6

Objectives

This programming project aims at exposing the students to the various aspects of programming language design and implementation by building a working compiler for a simple, but realistic high-level programming language. In this process, the students are expected to apply the knowledge acquired during the lectures and understand the various underlying algorithms and implementation trade-offs. The envisioned compiler will be able to handle a subset of the popular Java programming language and generate valid Java Virtual Machine (JVM) instructions in the *jasmin* format, which are then translated into Java *bytecodes* by the Jasmin assembler.

In the previous checkpoint, you focused on the generation of OLLIR code and optimizations for both AST and register base intermediate representations. In this checkpoint, you will implement the final stage in the compiler, the backend, by generating Jasmin code (a Java bytecode assembler).

Generate Jasmin code

The next step after the OO-based Low-Level Intermediate Representation (OLLIR) generation is the generation of the Jasmin code from OLLIR code. Similarly to what you did when converting from the AST to OLLIR code, now you will repeat the process, but starting from OLLIR's root node, the *ClassUnit* that you can access from an instance of *OllirResult*.

OLLIR does not provide visitors to its internal representation, but it has a tree structure where you can ask for the children of each element, which the class **JasminGenerator** uses to manually implement a visitor. You should expand on this class in order to generate the Jasmin code that is missing.

Check Jasmin's webpage [2] for further details on the Jasmin format and assembler operation, and Oracle's Java bytecodes documentation [4], which contains comprehensive and in-depth information about all available bytecodes. We also recommend Wikipedia's list of Java bytecode instructions [5] as a quick-reference.

When generating Jasmin code, you will need to convert the OLLIR types to Java bytecode type descriptors, you can check the Java documentation for details on the descriptor format [3].

1.1 Interfaces

You will output Jasmin code with the class *JasminBackendImpl*, which implements the interface *JasminBackend*. This interface contains a method *toJasmin* which converts an OLLIR *ClassUnit* to a String representing Jasmin bytecode. This method expects as input the result of the previous stage, a *OllirResult*, and returns a *JasminResult* instance, as follows:

```
JasminResult toJasmin(OllirResult ollirResult);
```

The output of this stage is an instance of the *JasminResult* class, which can be built using the previous *OllirResult*, a String with the generated Jasmin bytecode, and possibly additional reports. This result is used by the Jasmin tool to generate actual Java bytecode.

Note that now that you have finished the last stage of the compiler, you can actually execute the code, and the class *JasminResult* has methods for that (e.g., `run(List<String> args, List<String> classpath)`).

1.2 Always generate a public class

Even if your original class in JAVA-- is not declared as public, we ask you to always add the keyword `public` in the class you generate in Jasmin. This is due to testing purposes, so that we can automatically load and execute your class using reflection. The public tests will check if you are adding the public keyword.

1.3 Instruction Selections

The JVM has some instructions that are a specialization of other more generic instructions. For instance, you can use the instruction `ldc` to load any valid constant, but there are instructions for when the constant has a certain size (e.g., `sipush` when the constant fits in a short).

These specialized instructions usually have performance benefits (e.g., code size), and should always be used when possible, even when optimizations are disabled (i.e., the flag “-o”). The expected instructions that you should specialize include: `iload`, `istore`, `astore`, `aload`, loading constants to the stack, use of `iinc` and comparisons to zero (e.g., `iflt`).

1.4 `.limit` directives

For each JVM method, you have to add a preamble where you set the values for the directives `.limit stack` and `.limit locals`.

The value of `.limit locals` corresponds to the maximum number of registers required for that method. Take into account that:

- if you have an instance method, register 0 always contains `this`;
- an instruction such as `istore 5` implies at least 6 registers;
- the parameters of the method also count towards `.limit locals`.

The value of `.limit stack` corresponds to the maximum stack size that is needed for a given method. The stack contains all the arguments for invocations and other instructions. Keep in mind that, if you think back to the JAVA-- code, after executing any JAVA-- statement, the stack should be empty. If the statement contains multiple subexpressions, during its execution, the stack will hold the necessary intermediate values.

The initial version of the project sets the values of the limits to 99, you will have to calculate the correct values for each method.

1.5 When in doubt, reverse engineer it

You can use Java decompilers to generate as many examples of Java bytecodes as necessary [1]. JAVA-- was developed to be a subset of Java, so you can compile a `.jmm` file as if it was a Java file and then decompile it.

The main difference between pure Java bytecodes and Jasmin is that decompiled bytecodes use indexes to a constant pool for things like method descriptors. However, most decompilers will also provide information about what the constant pool index being used is referring to, usually as a string similar to what you have to generate in Jasmin.

Checklist for CP3

This section lists a set of tasks that work as guidelines for the development. You should divide the work among the group members.

Note: The following checklist applies to groups with 4 elements. If your group has fewer elements, you will not need to implement language features related to **fields** and **arrays**. This means that during the evaluation of your work, we will not apply tests that contain code with those elements.

2.1 Checklist for Core Features

2.1.1 Declarations

Jasmin Generation

- ☐ Class structure
 - * Including the `<init>` constructor
- ☐ Method structure
- ☐ Calculate `.limit locals` and `.limit stack`

2.1.2 Statements

Jasmin Generation

- ☐ Conditional instructions
- ☐ While Loops
- ☐ Return statements
- ☐ When appropriate, replace load and store instructions (e.g., `iload x`) with lower cost alternatives (e.g., `iload_x`, `istore_x`, `astore_x`, `aload_x`)
- ☐ When appropriate, replace branch statements that use two values (e.g., `if_icmplt`) with branch statements that compare against zero (e.g., `iflt`, `ifne`)

2.1.3 Entities and Operations

Jasmin Generation

- ☐ Class fields
- ☐ Assignments
- ☐ Arithmetic and logical operators
 - * With correct precedence
- ☐ When appropriate, replace use of `ldc` with lower cost alternatives (e.g., `iconst_0`, `bipush`, `sipush`)
- ☐ When appropriate, use `iinc` (e.g., replace `i=i+1` with `i++`)

2.1.4 Calls

Jasmin Generation

- ☐ Invocation of static methods
- ☐ Invocation of instance methods
- ☐ Invocation of new methods (without arguments)

2.1.5 Arrays

Jasmin Generation

- ☐ Array instantiation
- ☐ Reading elements from arrays
- ☐ Writing elements to arrays

2.2 Checklist for Extensions

2.2.1 Declarations

Arbitrary Position

Jasmin Generation

- ☐ Generation of class fields at the beginning of the class

Method Visibility

Jasmin Generation

- ☐ Correct generation of the visibility modifiers

Field Initializer

Jasmin Generation

- ☐ Generation of field initialization code inside class constructor

2.2.2 Statements

if without else

Jasmin Generation

- ☐ Correctly generate code that respects the original control flow

for loop statement

Jasmin Generation

- ☐ Correctly generate Jasmin code corresponding to for loop

do-while loop statement

Jasmin Generation

- ☐ Correctly generate Jasmin code corresponding to do-while loop

2.2.3 Entities and Operations

Additional Arithmetic operations

Jasmin Generation

- ☐ Additional arithmetic operators
 - * With correct precedence

Additional Logical Operations

Jasmin Generation

- ☐ Additional logical operators
 - * With correct precedence

2.2.4 Calls

Local Static Methods

Jasmin Generation

- ☐ Correct generation of static methods

New with Arguments

Jasmin Generation

- ☐ Invocation of new methods with arbitrary arguments

Implicit this

Jasmin Generation

- ☐ Invocation of instance methods of current object

2.2.5 Arrays

Array Initializer

Jasmin Generation

- ☐ Correctly generate Jasmin code corresponding to array initializers

Multidimensional Arrays

Jasmin Generation

- ☐ Multidimensional array instantiation
- ☐ Reading elements from multidimensional arrays
- ☐ Writing elements to multidimensional arrays

Bibliography

- [1] Compiler explorer. <https://godbolt.org/>.
- [2] Jasmin home page. <http://jasmin.sourceforge.net/>.
- [3] Java bytecode type descriptors. <https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-4.html#jvms-4.3.2-200>.
- [4] The java virtual machine instruction set. <https://docs.oracle.com/javase/specs/jvms/se7/html/jvms-6.html>.
- [5] List of java bytecode instructions. https://en.wikipedia.org/wiki/List_of_Java_bytecode_instructions.